

Algorytmy zaawansowane

Opracowanie zagadnień egzaminacyjnych
(algorytmy obowiązujące, twierdzenia z dowodami, projektowanie algorytmów)

Opracowanie na podstawie wykładów

17 czerwca 2026

Spis treści

Jak czytać to opracowanie	2
1 Narzędzia: strategie, asymptotyka, twierdzenie o rekurencji	2
1.1 Cztery strategie projektowe	2
1.2 Notacja asymptotyczna	2
1.3 Uniwersalne twierdzenie o rekurencji (Master Theorem)	3
1.4 Współczynnik aproksymacji	3
2 Para najbliższych punktów na płaszczyźnie	3
3 Matrix-Multiply i Matrix-Chain-Order	4
4 Najdłuższy wspólny podciąg (LCS)	5
5 Zachłanny wybór zajęć	6
6 Kody Huffmana	7
7 Szeregowanie zadań	8
7.1 Wariant A: minimalizacja średniego czasu zakończenia	8
7.2 Wariant B: zadania z terminami i karami — struktura matroidu	9
8 Greedy-Set-Cover	9
9 Approx-Vertex-Cover	10
10 Exact-Subset-Sum	11
11 Approx-Subset-Sum (FPTAS)	12
12 Maksymalne skojarzenie w grafie dwudzielnym (MM)	13
13 Zestawienie twierdzeń z dowodami	14
14 Schemat odpowiedzi i przykłady projektowe	15
14.1 Schemat poprawnej odpowiedzi	15
14.2 Przykład A (DP): ważony wybór zajęć	15
14.3 Przykład B (zachłanny + argument wymiany): minimalizacja maksymalnego opóź- nienia	15
14.4 Przykład C (D&Z): liczba inwersji	15

14.5 Przykład D (aproksymacja / FPTAS): problem plecakowy 0/1	16
14.6 Modyfikacje dla szczególnych danych (zadanie 3)	16
Podsumowanie złożoności	16

Jak czytać to opracowanie

Egzamin obejmuje cztery typy zadań:

1. **Sformułować i udowodnić twierdzenie** (pięć wskazanych twierdzeń) — patrz sekcja 13 oraz dowody wplecione w opisy algorytmów.
2. **Zaprezentować działanie algorytmu na konkretnych danych** — dla każdego z 11 algorytmów podano w pełni przeliczony przykład.
3. **Zaprojektować modyfikację algorytmu** dla szczególnych danych — sekcja 14.
4. **Zaprojektować algorytm** (dokładny / przybliżony / wg zadanej strategii, o zadanej złożoności), oszacować złożoność, udowodnić poprawność i współczynnik jakości — sekcja 14.

Dla każdego algorytmu konsekwentnie podajemy: *definicję problemu, ideę i strategię, pseudokod, złożoność, dowód poprawności oraz przykład.*

1 Narzędzia: strategię, asymptotyka, twierdzenie o rekurencji

1.1 Cztery strategie projektowe

- **Dziel i zwyciężaj.** Dzielimy problem na rozłączne podproblemy, rozwiązujemy je rekurencyjnie i łączymy wyniki. (Para najbliższych punktów, mnożenie macierzy/liczb.)
- **Programowanie dynamiczne.** Wymaga *własności optymalnej podstruktury* (rozwiązanie optymalne składa się z optymalnych rozwiązań podproblemów) oraz *nakładających się podproblemów* (mała, wielomianowa liczba różnych podproblemów). Budujemy rozwiązanie od dołu do góry lub przez spamiętywanie. (LCS, Matrix-Chain, Subset-Sum.)
- **Algorytmy zachłanne.** W każdym kroku wybór lokalnie najlepszy; poprawność wymaga *własności wyboru zachłannego* i *optymalnej podstruktury* (często: struktury matroidu). (Wybór zajęć, Huffman, szeregowanie zadań.)
- **Algorytmy aproksymacyjne.** Dla problemów trudnych (NP-trudnych) zwracamy rozwiązanie *prawie optymalne* z gwarantowanym współczynnikiem jakości. (Approx-Vertex-Cover, Greedy-Set-Cover, Approx-Subset-Sum.)

1.2 Notacja asymptotyczna

Dla $f, g : \mathbb{N} \rightarrow \mathbb{R}_+$:

$$g = O(f) \iff \exists c > 0 \exists n_0 \forall n \geq n_0 : g(n) \leq cf(n); \quad g = \Omega(f) \iff f = O(g); \quad g = \Theta(f) \iff g = O(f) \wedge g = \Omega(f)$$

1.3 Uniwersalne twierdzenie o rekurencji (Master Theorem)

Twierdzenie 1.1 (Master Theorem). Niech $a \geq 1$, $b > 1$ stałe i $T(n) = aT(n/b) + f(n)$. Wtedy

$$T(n) = \begin{cases} \Theta(n^{\log_b a}), & f(n) = O(n^{\log_b a - \varepsilon}), \varepsilon > 0, \\ \Theta(n^{\log_b a} \log n), & f(n) = \Theta(n^{\log_b a}), \\ \Theta(f(n)), & f(n) = \Omega(n^{\log_b a + \varepsilon}), \varepsilon > 0, \text{ oraz } af(n/b) \leq cf(n), c < 1. \end{cases}$$

W szczególności, jeśli $f(n) = \Theta(n^k)$, to porównujemy $\log_b a$ z k : $T(n) = \Theta(n^{\log_b a})$ gdy $\log_b a > k$; $\Theta(n^k \log n)$ gdy $\log_b a = k$; $\Theta(n^k)$ gdy $\log_b a < k$.

Przykłady użycia: para najbliższych punktów $T(n) = 2T(n/2) + O(n) = \Theta(n \log n)$; Strassen $T(n) = 7T(n/2) + \Theta(n^2) = \Theta(n^{\log_2 7}) = \Theta(n^{2.81})$.

1.4 Współczynnik aproksymacji

Definicja 1.2 (współczynnik aproksymacji). Dla problemu optymalizacyjnego niech $c(I)$ będzie wartością rozwiązania zwróconego przez algorytm A na instancji I , a $c^*(I)$ wartością optymalną. Algorytm A ma *współczynnik aproksymacji* $\rho(n)$, jeśli dla każdej instancji rozmiaru n

$$\max \left\{ \frac{c(I)}{c^*(I)}, \frac{c^*(I)}{c(I)} \right\} \leq \rho(n).$$

Konwencja daje $\rho(n) \geq 1$: dla minimalizacji $c \geq c^*$, więc $\rho = c/c^*$; dla maksymalizacji $c \leq c^*$, więc $\rho = c^*/c$. $\rho = 1$ oznacza algorytm dokładny.

Część A. Algorytmy obowiązujące

2 Para najbliższych punktów na płaszczyźnie

Strategia: dziel i zwyciężaj. **Złożoność:** $\Theta(n \log n)$.

Problem

Dany zbiór Q z $n \geq 2$ punktów na płaszczyźnie. Znaleźć parę punktów o najmniejszej odległości euklidesowej $d(p_1, p_2) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. Metoda siłowa: $\binom{n}{2} = \Theta(n^2)$.

Idea algorytmu

W każdym wywołaniu mamy zbiór P oraz tablice X (punkty P posortowane po x) i Y (posortowane po y). Jeśli $|P| \leq 3$ — sprawdzamy wszystkie pary.

Dziel

Pionowa prosta l dzieli P na P_L (lewe, $\lceil |P|/2 \rceil$) i P_R (prawe, $\lfloor |P|/2 \rfloor$). Tablice X, Y dzielimy w czasie $O(n)$ na X_L, X_R, Y_L, Y_R z zachowaniem posortowania.

Zwyciężaj

Rekurencyjnie liczymy δ_L, δ_R ; niech $\delta = \min(\delta_L, \delta_R)$.

Połącz

Para o odległości $< \delta$ łącząca strony leży w pasie szerokości 2δ wokół l . Tworzymy Y' (punkty z pasa, posortowane po y) i dla każdego $p \in Y'$ sprawdzamy odległość do **7 kolejnych** punktów w Y' . Jeśli znaleziono parę $< \delta$, zwracamy ją, w przeciwnym razie δ .

Złożoność

Po jednorazowym wstępnym sortowaniu ($O(n \log n)$): $T(n) = 2T(n/2) + O(n) = \Theta(n \log n)$ (Tw. 1.1). Łącznie $O(n \log n)$.

Poprawność

Twierdzenie 2.1 (poprawność algorytmu pary najbliższych punktów). *W fazie Połącz wystarczy dla każdego $p \in Y'$ porównać p z jego 7 kolejnymi punktami w Y' , aby na pewno znaleźć parę najbliższą, jeżeli realizuje ją para typu „lewy-prawy”.*

Dowód. Niech p, q będzie parą najbliższą i $d(p, q) = \delta' < \delta$, $p = (x_p, y_p)$, $q = (x_q, y_q)$, $y_p \leq y_q$. Oba punkty leżą w pasie, więc są w Y' . Ponieważ $y_q - y_p \leq d(p, q) < \delta$, wszystkie punkty Y' „między” p a q (włącznie) leżą w prostokącie $2\delta \times \delta$ (szerokość pasa 2δ , wysokość $< \delta$). Podzielmy go na lewy i prawy kwadrat $\delta \times \delta$. W jednym kwadracie $\delta \times \delta$ wszystkie punkty pochodzą z tej samej strony (P_L lub P_R), a tam minimalna odległość to $\geq \delta$. W kwadracie $\delta \times \delta$ nie da się rozmieścić więcej niż 4 punktów parami oddalonych o $\geq \delta$, więc w każdym kwadracie jest ≤ 4 punktów, łącznie ≤ 8 w prostokącie. Zatem między p a q w Y' jest co najwyżej 6 punktów, czyli q jest jednym z 7 kolejnych po p . Wystarczy więc sprawdzić 7 następników. $\square$

Przykład

$Q = \{(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)\}$. Punkty $(2, 3)$ i $(3, 4)$ dają $d = \sqrt{1+1} = \sqrt{2} \approx 1,41$ — najmniejsza odległość. Linia podziału rozdziela zbiór; w lewej części $\{(2, 3), (5, 1), (3, 4)\}$ sprawdzenie wszystkich par (bo ≤ 3) zwraca $\delta_L = \sqrt{2}$; prawa część daje większe δ_R ; w pasie szerokości $2\sqrt{2}$ nie pojawia się mniejsza para, więc wynik to $((2, 3), (3, 4))$ z odległością $\sqrt{2}$.

3 Matrix-Multiply i Matrix-Chain-Order

Strategia: (Strassen) dziel i zwyciężaj; (nawiasowanie) programowanie dynamiczne.

Mnożenie macierzy $A \cdot B$ ($n \times n$)

Klasycznie $\Theta(n^3)$. Dzieląc na bloki $\frac{n}{2} \times \frac{n}{2}$ naiwnie: $T(n) = 8T(n/2) + \Theta(n^2) = \Theta(n^3)$. **Strassen** liczy 7 iloczynów $M_1, \dots, M_7$:

$$\begin{aligned} M_1 &= (A_{12} - A_{22})(B_{21} + B_{22}), & M_2 &= (A_{11} + A_{22})(B_{11} + B_{22}), & M_3 &= (A_{11} - A_{21})(B_{11} + B_{12}), \\ M_4 &= (A_{11} + A_{12})B_{22}, & M_5 &= A_{11}(B_{12} - B_{22}), & M_6 &= A_{22}(B_{21} - B_{11}), \\ M_7 &= (A_{21} + A_{22})B_{11}, \end{aligned}$$

$$\begin{aligned} C_{11} &= M_1 + M_2 - M_4 + M_6, & C_{12} &= M_4 + M_5, & C_{21} &= M_6 + M_7, & C_{22} &= M_2 - M_3 + M_5 - M_7. \\ T(n) &= 7T(n/2) + \Theta(n^2) = \Theta(n^{\log_2 7}) = \Theta(n^{2,81}). \end{aligned}$$

Optymalne nawiasowanie ciągu macierzy

Wymiary $p = (p_0, \dots, p_n)$, A_i ma wymiar $p_{i-1} \times p_i$. Mnożenie $p \times q$ przez $q \times r$ kosztuje pqr . Mnożenie jest łączne; szukamy nawiasowania o najmniejszej liczbie mnożeń skalarnych.

Optymalna podstruktura.

Twierdzenie 3.1. *Jeśli optymalne nawiasowanie $A_i \cdots A_j$ wykonuje ostatnie mnożenie między A_k a A_{k+1} , to nawiasowania podciągów $A_i \cdots A_k$ oraz $A_{k+1} \cdots A_j$ są optymalne.*

Dowód. Gdyby lewy podciąg dało się nawiasować taniej, podmiana zmniejszyłaby koszt całości (prawa część i ostatnie mnożenie bez zmian) — sprzeczność z optymalnością. Analogicznie dla prawego. $\square$

Rekurencja.

$$m[i, j] = \begin{cases} 0, & i = j, \\ \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + p_{i-1}p_kp_j\}, & i < j, \end{cases} \quad s[i, j] = \arg \min.$$

Algorytm 1 Matrix-Chain-Order(p)

```
1:  $n \leftarrow \text{length}(p) - 1$ 
2: dla  $i = 1$  to  $n$  wykonuj  $m[i, i] \leftarrow 0$ 
3: koniec dla
4: dla  $\ell = 2$  to  $n$  wykonuj  $\triangleright \ell = \text{długość podciągu}$ 
5:   dla  $i = 1$  to  $n - \ell + 1$  wykonuj
6:      $j \leftarrow i + \ell - 1$ ;  $m[i, j] \leftarrow \infty$ 
7:     dla  $k = i$  to  $j - 1$  wykonuj
8:        $q \leftarrow m[i, k] + m[k+1, j] + p_{i-1}p_kp_j$ 
9:       jeżeli  $q < m[i, j]$  to  $m[i, j] \leftarrow q$ ;  $s[i, j] \leftarrow k$ 
10:      koniec jeżeli
11:    koniec dla
12:  koniec dla
13: koniec dla
14: zwróć ( $m, s$ )
```

Złożoność: $\Theta(n^2)$ podproblemów $\times O(n)$ podziałów = $\Theta(n^3)$ czasu, $\Theta(n^2)$ pamięci. Odtworzenie nawiasowania: rekurencyjnie z s (PRINT-OPTIMAL-PARENS).

Przykład

$p = (10, 100, 5, 50)$, czyli $A_1: 10 \times 100$, $A_2: 100 \times 5$, $A_3: 5 \times 50$. Dwa nawiasowania: $((A_1A_2)A_3): 10 \cdot 100 \cdot 5 + 10 \cdot 5 \cdot 50 = 5000 + 2500 = 7500$; $(A_1(A_2A_3)): 100 \cdot 5 \cdot 50 + 10 \cdot 100 \cdot 50 = 25000 + 50000 = 75000$. Optymalne to $((A_1A_2)A_3)$ z kosztem 7500 (tu $m[1, 3] = 7500$, $s[1, 3] = 2$).

4 Najdłuższy wspólny podciąg (LCS)

Strategia: programowanie dynamiczne. **Złożoność:** $\Theta(mn)$.

Problem

Z jest *podciągiem* $X = (x_1, \dots, x_m)$, jeśli istnieją indeksy $i_1 < \dots < i_k$ z $z_j = x_{i_j}$. Dla X, Y szukamy najdłuższego wspólnego podciągu. Przeszukiwanie wszystkich 2^m podciągów — wykładnicze. Oznaczamy $X_i = (x_1, \dots, x_i)$, $c[i, j] = |\text{LCS}(X_i, Y_j)|$.

Optymalna podstruktura

Twierdzenie 4.1 (struktura LCS). *Niech $Z = (z_1, \dots, z_k)$ będzie LCS ciągów X, Y .*

- (1) *Jeśli $x_m = y_n$, to $z_k = x_m = y_n$ oraz Z_{k-1} jest LCS X_{m-1} i Y_{n-1} .*
- (2) *Jeśli $x_m \neq y_n$ i $z_k \neq x_m$, to Z jest LCS X_{m-1} i Y .*
- (3) *Jeśli $x_m \neq y_n$ i $z_k \neq y_n$, to Z jest LCS X i Y_{n-1} .*

Dowód. (1) Gdyby $z_k \neq x_m$, można by dopisać $x_m = y_n$ do Z , tworząc wspólny podciąg długości $k+1$ — sprzeczność, więc $z_k = x_m = y_n$. Ciąg Z_{k-1} jest wspólnym podciągiem X_{m-1}, Y_{n-1} ; gdyby istniał dłuższy W , to W z dopisanym x_m byłby wspólnym podciągiem X, Y dłuższym niż Z — sprzeczność. (2),(3) Jeśli $z_k \neq x_m$, to Z jest podciągiem X_{m-1} i Y ; każdy dłuższy ich wspólny podciąg byłby też wspólnym podciągiem X, Y — sprzeczność. Przypadek (3) analogicznie. $\square$

$$c[i, j] = \begin{cases} 0, & i = 0 \text{ lub } j = 0, \\ c[i-1, j-1] + 1, & i, j > 0, x_i = y_j, \\ \max\{c[i-1, j], c[i, j-1]\}, & i, j > 0, x_i \neq y_j. \end{cases}$$

Algorytm 2 LCS(X, Y)

```

1:  $m \leftarrow \text{length}(X)$ ;  $n \leftarrow \text{length}(Y)$ 
2: dla  $i = 0$  to  $m$  wykonuj  $c[i, 0] \leftarrow 0$ 
3: koniec dla
4: dla  $j = 0$  to  $n$  wykonuj  $c[0, j] \leftarrow 0$ 
5: koniec dla
6: dla  $i = 1$  to  $m$  wykonuj
7:   dla  $j = 1$  to  $n$  wykonuj
8:     jeżeli  $x_i = y_j$  to  $c[i, j] \leftarrow c[i-1, j-1] + 1$ ;  $\ell[i, j] \leftarrow \nwarrow$ 
9:     w przeciwnym razie jeżeli  $c[i-1, j] \geq c[i, j-1]$  to  $c[i, j] \leftarrow c[i-1, j]$ ;  $\ell[i, j] \leftarrow \uparrow$ 
10:    w przeciwnym razie  $c[i, j] \leftarrow c[i, j-1]$ ;  $\ell[i, j] \leftarrow \leftarrow$ 
11:    koniec jeżeli
12:  koniec dla
13: koniec dla
14: zwróć  $(c, \ell)$ 

```

Czas i pamięć $\Theta(mn)$; odtworzenie (PRINT-LCS) idzie po strzałkach ℓ w czasie $O(m+n)$.

Przykład

$X = (A, B, C, B, D, A, B)$, $Y = (B, D, C, A, B, A)$. Tablica c daje $c[7, 6] = 4$, a odtworzenie zwraca $\text{LCS} = (B, C, B, A)$ długości 4.

5 Zachłanny wybór zajęć

Strategia: zachłanna. **Złożoność:** $O(n \log n)$ (z sortowaniem).

Problem

Zajęcia $z_i = (s_i, t_i)$, $s_i < t_i$, jeden zasób. Zajęcia z_i, z_j *zgodne*, gdy przedziały rozłączne. Szukamy najliczniejszego podzbioru parami zgodnych.

Idea i algorytm

Sortujemy rosnąco po czasie zakończenia $t_1 \leq \dots \leq t_n$ i zachłannie bierzemy zajęcie kończące się najwcześniej, a potem każde kolejne zgodne z ostatnio wybranym.

Algorytm 3 Activity-Selection(s, t) (po posortowaniu wg t)

```
1:  $A \leftarrow \{z_1\}$ ;  $j \leftarrow 1$ 
2: dla  $i = 2$  to  $n$  wykonuj
3:   jeżeli  $s_i \geq t_j$  to  $A \leftarrow A \cup \{z_i\}$ ;  $j \leftarrow i$ 
4:   koniec jeżeli
5: koniec dla
6: zwróć  $A$ 
```

Poprawność

Lemat 5.1 (wybór zachłanny). *Niech z_m ma najwcześniejszy czas zakończenia spośród rozważanych zajęć. Wtedy istnieje optymalne rozwiązanie zawierające z_m .*

Dowód. Niech A^* będzie optymalne i z_a jego zajęciem o najwcześniejszym końcu. Ponieważ $t_m \leq t_a$, zamiana $z_a \rightarrow z_m$ daje zbiór $A^* \setminus \{z_a\} \cup \{z_m\}$ tej samej liczności; z_m jest zgodne z resztą (kończy się nie później niż z_a , a pozostałe zaczynają się po $t_a \geq t_m$). Zatem to też rozwiązanie optymalne, zawierające z_m . $\square$

Twierdzenie 5.2. *Algorytm zwraca rozwiązanie optymalne.*

Dowód. Indukcja po n . Z Lematu 5.1 z_1 należy do pewnego A^* . Po wybraniu z_1 zostaje podproblem $S' = \{z_k : s_k \geq t_1\}$; $A^* \setminus \{z_1\}$ jest dla niego optymalne (gdyby istniało lepsze B , to $\{z_1\} \cup B$ byłoby A^*). Z założenia indukcyjnego algorytm rozwiązuje S' optymalnie, więc $\{z_1\} \cup$ (rozw. S') jest optymalne dla S . $\square$

Przykład

i	1	2	3	4	5	6
s_i	1	0	2	4	7	7
t_i	3	4	6	7	9	10

Po sortowaniu wg t : bierzemy z_1 ($t = 3$); z_4 ($s = 4 \geq 3$); z_5 ($s = 7 \geq 7$). Wynik $A = \{z_1, z_4, z_5\}$, liczność 3.

6 Kody Huffmana

Strategia: zachłanna. **Złożoność:** $O(n \log n)$.

Problem

Alfabet C , częstości $f : C \rightarrow \mathbb{N}$. Kod prefiksowy $\leftrightarrow$ regularne drzewo binarne (znaki w liściach). Koszt $B(T) = \sum_{c \in C} f(c) d_T(c)$, gdzie $d_T(c)$ to głębokość liścia. Szukamy drzewa minimalizującego $B(T)$.

Algorytm

Algorytm 4 Huffman(C, f)

```
1:  $Q \leftarrow$  kolejka priorytetowa liści  $c \in C$  z kluczem  $f(c)$ 
2: dla  $i = 1$  to  $|C| - 1$  wykonuj
3:    $x \leftarrow \text{EXTRACT-MIN}(Q)$ ;  $y \leftarrow \text{EXTRACT-MIN}(Q)$ 
4:   utwórz węzeł  $z$  z dziećmi  $x, y$ ;  $f(z) \leftarrow f(x) + f(y)$ ;  $\text{INSERT}(Q, z)$ 
5: koniec dla
6: zwróć  $\text{EXTRACT-MIN}(Q)$  ▷ korzeń
```

Na kopcu binarnym każda operacja $O(\log n)$, więc łącznie $O(n \log n)$.

Poprawność

Lemat 6.1. *Niech x, y to dwa znaki o najmniejszych częstościach. Istnieje optymalny kod, w którym x, y są rodzeństwem na największej głębokości.*

Dowód. Niech T będzie optymalne, a a, b — rodzeństwo o największej głębokości; BSO $f(x) \leq f(y)$, $f(a) \leq f(b)$, przy czym $f(x) \leq f(a)$, $f(y) \leq f(b)$. Zamiana liści $x \leftrightarrow a$ daje T' z

$$B(T) - B(T') = (f(a) - f(x))(d_T(a) - d_T(x)) \geq 0,$$

bo oba czynniki ≥ 0 (a jest najgłębsze). Skoro T optymalne, $B(T') = B(T)$. Analogicznie $y \leftrightarrow b$. Otrzymane drzewo jest optymalne i ma x, y jako rodzeństwo najgłębiej. $\square$

Lemat 6.2. *Niech $A' = (A \setminus \{x, y\}) \cup \{z\}$, $f'(c) = f(c)$ dla $c \neq z$, $f'(z) = f(x) + f(y)$. Jeśli T' jest optymalne dla (A', f') , to T powstałe przez zastąpienie liścia z węzłem z dziećmi x, y jest optymalne dla (A, f) .*

Dowód. Z $d_T(x) = d_T(y) = d_{T'}(z) + 1$ wynika rachunkiem $B(T) = B(T') + f(x) + f(y)$. Gdyby istniało T_{opt} dla (A, f) z $B(T_{\text{opt}}) < B(T)$, to (na mocy Lem. 6.1 z x, y jako rodzeństwem) sklejenie x, y w liść z daje drzewo dla (A', f') o koszcie $B(T_{\text{opt}}) - f(x) - f(y) < B(T) - f(x) - f(y) = B(T')$ — sprzeczność z optymalnością T' . $\square$

Twierdzenie 6.3. *Algorytm Huffmana konstruuje optymalny kod prefiksowy.*

Dowód. Indukcja po $|C|$, krok wprost z Lematów 6.1 i 6.2 (pierwsze sklejenie łączy dwa najmniejsze znaki, a na mniejszym alfabecie algorytm jest optymalny z zał. indukcyjnego). $\square$

Przykład

Częstości: $n:1$, $s:3$, $t:4$, $a:10$, $c:12$, $r:13$, $e:15$. Sklejenia: $1+3=4$, $4+4=8$, $8+10=18$, $12+13=25$, $18+15=33$, $25+33=58$. Słowa kodowe (np.): $c=00$, $r=01$, $e=10$, $a=111$, $t=1101$, $n=11000$, $s=11001$. Koszt = $12 \cdot 2 + 13 \cdot 2 + 15 \cdot 2 + 10 \cdot 3 + 4 \cdot 4 + 1 \cdot 5 + 3 \cdot 5 = 146$ bitów (kod o stałej długości: $3 \cdot 58 = 174$).

7 Szeregowanie zadań

Dwa warianty: (A) minimalizacja *średniego czasu zakończenia* (zachłanny SPT) oraz (B) *zadania z terminami i karami* (struktura matroidu) — to drugie jest źródłem twierdzenia egzaminacyjnego „o strukturze problemu szeregowania”.

7.1 Wariant A: minimalizacja średniego czasu zakończenia

Zadania z_i o czasach t_i , jeden procesor. Dla kolejności $j_1, \dots, j_n$ czas zakończenia $C_k = \sum_{l=1}^k t_{j_l}$, minimalizujemy $\frac{1}{n} \sum_k C_k$.

Twierdzenie 7.1 (SPT). *Uszeregowanie w kolejności niemalejących czasów ($t_{j_1} \leq \dots \leq t_{j_n}$) minimalizuje średni czas zakończenia.*

Dowód (argument wymiany). Jeśli optymalna permutacja nie jest posortowana, istnieją sąsiednie j_k, j_{k+1} z $t_{j_k} > t_{j_{k+1}}$. Zamiana zmienia jedynie C_k i daje $f(\Pi') - f(\Pi) = \frac{t_{j_{k+1}} - t_{j_k}}{n} \leq 0$, więc Π' też optymalna. Po skończeniu wielu zamianach dostajemy permutację posortowaną. $\square$

Dla m procesorów: przydzielaj zadania od najkrótszego do najdłuższego do pierwszego wolnego procesora (ten sam argument wymiany).

Przykład. $t = (15, 8, 3, 10)$. Kolejność SPT z_3, z_2, z_4, z_1 daje $\frac{3+11+21+36}{4} = 17,75$ wobec 25 dla kolejności pierwotnej.

7.2 Wariant B: zadania z terminami i karami — struktura matroidu

Zadania $S = \{z_1, \dots, z_n\}$ o jednostkowym czasie, terminy $d_i \in [n]$, kary $w_i \geq 0$. Karę w_i płacimy, gdy z_i nie zakończy się do chwili d_i (jest *spóźnione*). Minimalizujemy sumę kar. Zbiór $A \subseteq S$ jest *wolny od kar*, gdy istnieje harmonogram, w którym wszystkie zadania A są terminowe. Niech $N_t(A) = |\{z_i \in A : d_i \leq t\}|$.

Lemat 7.2 (charakteryzacja, bez dowodu na egzaminie). Dla $A \subseteq S$ równoważne są: (1) A jest wolny od kar; (2) $N_t(A) \leq t$ dla każdego $t \in \{1, \dots, n\}$; (3) ustawiając A niemalejąco wg terminów, żadne zadanie nie jest spóźnione.

Twierdzenie 7.3 (struktura problemu szeregowania). Niech $\mathcal{A}$ będzie rodziną wszystkich zbiorów wolnych od kar. Wtedy $(S, \mathcal{A})$ jest matroidem.

Dowód. Sprawdzamy cztery warunki definicji matroidu. (Skończoność) S skończony. (Niepustość/dziedziczność dla $\emptyset$) $\emptyset \in \mathcal{A}$, a każde $\{z_i\}$ jest wolne od kar. (Dziedziczność) Jeśli $A \in \mathcal{A}$ i $B \subseteq A$: z harmonogramu dla A (wszystkie terminowe) usuwamy zadania spoza B — pozostałe nadal terminowe, więc $B \in \mathcal{A}$. (Własność wymiany) Niech $A, B \in \mathcal{A}$, $|B| > |A|$. Niech k będzie największym $t \in \{0, \dots, n\}$ z $N_t(B) \leq N_t(A)$ (poprawne, bo $N_0 = 0$). Ponieważ $N_n(B) = |B| > |A| = N_n(A)$, mamy $k < n$ oraz $N_{k+1}(B) > N_{k+1}(A)$, więc istnieje zadanie $z \in B \setminus A$ o terminie $d_z = k + 1$. Niech $A' = A \cup \{z\}$. Dla $t \leq k$: $N_t(A') = N_t(A) \leq t$. Dla $t \geq k + 1$: $N_t(A') = N_t(A) + 1 \leq N_t(B) \leq t$. Zatem (Lem. 7.2(2)) A' jest wolny od kar, czyli $A \cup \{z\} \in \mathcal{A}$. $\square$

Wniosek 7.4 (Rado–Edmonds). Ponieważ $(S, \mathcal{A})$ jest matroidem, algorytm zachłanny (sortuj zadania nierosnąco wg kar w_i i dodawaj z_i , o ile zbiór pozostaje wolny od kar) zwraca zbiór terminowy o maksymalnej sumie wag, a więc harmonogram o minimalnej sumie kar. Złożoność $O(n^2)$ (sprawdzenie warunku $O(n)$ na zadanie).

Przykład. $d = (3, 2, 2, 3, 1, 4, 6, 8)$ (dla $z_1, \dots, z_8$), $w = (80, 70, 60, 50, 40, 30, 20, 10)$. Przetwarzając wg malejących kar: bierzemy z_1, z_2, z_3 ; odrzucamy z_4 (N_3 wzrosłoby do $4 > 3$) i z_5 ($N_2 \rightarrow 3 > 2$); bierzemy z_6, z_7, z_8 . Zbiór terminowy $\{z_1, z_2, z_3, z_6, z_7, z_8\}$, spóźnione z_4, z_5 , suma kar $50 + 40 = 90$. Harmonogram $(z_2, z_3, z_1, z_6, z_7, z_8 \mid z_4, z_5)$.

8 Greedy-Set-Cover

Strategia: zachłanna/aproksymacyjna. **Współczynnik:** $H(\max |S|) \leq \ln n + 1$.

Problem

Dany zbiór bazowy X ($|X| = n$) i rodzina $\mathcal{F} \subseteq 2^X$ z $\bigcup_{S \in \mathcal{F}} S = X$. Szukamy najmniejszej podrodziny $\mathcal{C} \subseteq \mathcal{F}$ pokrywającej X . Problem NP-trudny.

Algorytm

Algorytm 5 Greedy-Set-Cover($X, \mathcal{F}$)

- 1: $U \leftarrow X; \mathcal{C} \leftarrow \emptyset$
 - 2: **dopóki** $U \neq \emptyset$ **wykonuj**
 - 3: wybierz $S \in \mathcal{F}$ maksymalizujące $|S \cap U|$ $\triangleright$ najwięcej nowych elementów
 - 4: $U \leftarrow U \setminus S; \mathcal{C} \leftarrow \mathcal{C} \cup \{S\}$
 - 5: **koniec dopóki**
 - 6: **zwróć** $\mathcal{C}$
-

Złożoność: $O(\sum_S |S|) = O(|X| |\mathcal{F}| \min(|X|, |\mathcal{F}|))$ — wielomianowa.

Współczynnik jakości

Niech $H(d) = \sum_{i=1}^d \frac{1}{i}$ ($H(0) = 0$), $H(d) \leq \ln d + 1$.

Twierdzenie 8.1 (jakość Greedy-Set-Cover). *Greedy-Set-Cover jest algorytmem $H(\max_{S \in \mathcal{F}} |S|)$ -aproxymacyjnym; w szczególności $H(\max |S|) \leq \ln n + 1$, więc współczynnik jest $O(\ln n)$.*

Dowód. Każdemu elementowi $x \in X$ przypisujemy koszt c_x : gdy zbiór S_i wybrany w i -tej iteracji pokrywa po raz pierwszy elementy zbioru U_i (tych jeszcze niepokrytych w S_i), to każdemu z nich nadajemy $c_x = \frac{1}{|S_i \cap U_i|}$. W każdej iteracji rozdzielamy łącznie koszt 1, więc

$$|\mathcal{C}| = \sum_{x \in X} c_x.$$

Niech $\mathcal{C}^*$ będzie pokryciem optymalnym. Ponieważ $\mathcal{C}^*$ pokrywa X ,

$$|\mathcal{C}| = \sum_{x \in X} c_x \leq \sum_{S \in \mathcal{C}^*} \sum_{x \in S} c_x.$$

Wystarczy pokazać, że dla każdego $S \in \mathcal{F}$ zachodzi $\sum_{x \in S} c_x \leq H(|S|)$. Ustalmy S i niech $u_i = |S \setminus (S_1 \cup \dots \cup S_i)|$ będzie liczbą elementów S niepokrytych po i iteracjach zachłannego algorytmu ($u_0 = |S|$). Niech k będzie pierwszą iteracją z $u_k = 0$. W iteracji i zachłanny wybór S_i pokrywa co najmniej tyle nowych elementów co S , czyli $|S_i \cap U_i| \geq u_{i-1}$ (bo S jest kandydatem pokrywającym u_{i-1} niepokrytych elementów). Każdemu z $u_{i-1} - u_i$ elementów S pokrytych w iteracji i przypisano koszt $\frac{1}{|S_i \cap U_i|} \leq \frac{1}{u_{i-1}}$. Stąd

$$\sum_{x \in S} c_x \leq \sum_{i=1}^k (u_{i-1} - u_i) \cdot \frac{1}{u_{i-1}} \leq \sum_{i=1}^k \sum_{j=u_i+1}^{u_{i-1}} \frac{1}{j} = \sum_{j=1}^{|S|} \frac{1}{j} = H(|S|),$$

gdzie skorzystaliśmy z $\frac{1}{u_{i-1}} \leq \frac{1}{j}$ dla $j \leq u_{i-1}$. Zatem $|\mathcal{C}| \leq \sum_{S \in \mathcal{C}^*} H(|S|) \leq |\mathcal{C}^*| H(\max |S|)$. $\square$

Przykład

$X = \{1, \dots, 12\}$, $\mathcal{F} = \{S_1, \dots\}$ z $S_1 = \{1, 2, 3, 4, 5, 6\}$, $S_2 = \{5, 6, 7, 8, 9, 10\}$, $S_3 = \{1, 2, 3, 4, 5, 6, 7\}$? — klasyczny przykład: pewne dwa duże zbiory pokrywają wszystko, ale zachłanny wybiera najpierw zbiór 6-elementowy, potem coraz mniejsze, zwracając 3 zbiory zamiast 2. Ilustruje to, że zachłanny nie jest dokładny, lecz $\leq H(6) = 1 + \frac{1}{2} + \dots + \frac{1}{6} \approx 2,45$ razy gorszy od optimum.

9 Approx-Vertex-Cover

Strategia: aproxymacyjna. **Współczynnik:** 2.

Problem

$G = (V, E)$. Pokrycie wierzchołkowe to $W \subseteq V$ takie, że każda krawędź ma koniec w W . Szukamy pokrycia o minimalnej liczności (NP-trudne).

Algorytm

Algorytm 6 Approx-Vertex-Cover(G)

```
1:  $C \leftarrow \emptyset$ ;  $F \leftarrow E(G)$ 
2: dopóki  $F \neq \emptyset$  wykonuj
3:   wybierz dowolną krawędź  $(u, v) \in F$ 
4:    $C \leftarrow C \cup \{u, v\}$ 
5:   usuń z  $F$  wszystkie krawędzie incydentne do  $u$  lub  $v$ 
6: koniec dopóki
7: zwróć  $C$ 
```

Złożoność $O(V + E)$ (przy listach sąsiedztwa).

Poprawność i współczynnik jakości

Twierdzenie 9.1 (jakość Approx-Vertex-Cover). *Approx-Vertex-Cover zwraca pokrycie wierzchołkowe o liczności $\leq 2|C^*|$, gdzie C^* jest pokryciem minimalnym; jest więc algorytmem 2-aproksymacyjnym.*

Dowód. Poprawność: pętla działa, dopóki istnieje niepokryta krawędź, a dodanie obu końców pokrywa ją; po zakończeniu $F = \emptyset$, więc C pokrywa wszystkie krawędzie.

Współczynnik: niech A będzie zbiorem krawędzi wybranych w wierszu „wybierz krawędź”. Po wybraniu (u, v) usuwamy wszystkie krawędzie incydentne do u i v , więc żadne dwie krawędzie z A nie dzielą końca — A jest **skojarzeniem**. Mamy $|C| = 2|A|$. Każde pokrycie C^* musi zawierać co najmniej jeden koniec każdej krawędzi z A , a krawędzie te są wierzchołkowo rozłączne, więc potrzeba $\geq |A|$ różnych wierzchołków: $|C^*| \geq |A|$. Stąd

$$|C| = 2|A| \leq 2|C^*|. \quad \square$$

Przykład

Graf o wierzchołkach a, b, c, d, e, f . Kolejne wybory krawędzi $(a, b), (c, d), (e, f)$ dają $C = \{a, b, c, d, e, f\}$ (6 wierzchołków). Skojarzenie $A = \{(a, b), (c, d), (e, f)\}$ ma 3 krawędzie, więc $\text{OPT} \geq 3$, a $|C| = 6 \leq 2 \cdot \text{OPT}$. (Pokrycie optymalne tego grafu, np. $\{a, d, e, f\}$, ma 4 wierzchołki.)

10 Exact-Subset-Sum

Strategia: programowanie dynamiczne (listy sum). **Złożoność:** wykładnicza w najgorszym razie, wielomianowa, gdy t wielomianowe.

Problem

Dany $S = \{x_1, \dots, x_n\} \subseteq \mathbb{N}$ i $t \in \mathbb{N}$. Wersja optymalizacyjna: znaleźć $S' \subseteq S$ o największej sumie $\leq t$. Oznaczenie: $P + x = \{s + x : s \in P\}$.

Lemat 10.1. *Niech $P_0 = \{0\}$, $P_i = P_{i-1} \cup (P_{i-1} + x_i)$. Wtedy P_n to zbiór wszystkich sum podzbiorów S .*

Dowód. Indukcja po i : P_i to wszystkie sumy podzbiorów $\{x_1, \dots, x_i\}$. Baza $P_0 = \{0\}$ (suma $\emptyset$).
 Krok: podzbiór bez x_i daje sumę z P_{i-1} ; z x_i — sumę z $P_{i-1} + x_i$. $\square$

Algorytm 7 Exact-Subset-Sum(S, t)

```

1:  $L_0 \leftarrow (0)$ 
2: dla  $i = 1$  to  $n$  wykonuj
3:    $L_i \leftarrow \text{MERGE-LISTS}(L_{i-1}, L_{i-1} + x_i)$   $\triangleright$  scalanie z usuwaniem duplikatów
4:   usuń z  $L_i$  elementy  $> t$ 
5: koniec dla
6: zwróć największy element  $L_n$ 
  
```

MERGE-LISTS scala dwie posortowane listy w $\Theta(|L| + |L'|)$. Lista L_i może mieć 2^i elementów — algorytm wykładniczy. Gdy jednak $t = O(n^p)$, długości $L_i \leq t + 1$ i algorytm jest wielomianowy.

Twierdzenie 10.2. *Problem sumy podzbiorów jest NP-zupełny (redukcja $3\text{-SAT} \leq_p \text{SUBSET-SUM}$).*

Przykład

$S = \{1, 4, 5\}$, $t = 10$. $L_0 = (0)$; $L_1 = (0, 1)$; $L_2 = (0, 1, 4, 5)$; $L_3 = (0, 1, 4, 5, 6, 9, 10)$ (po dodaniu 5 i odcięciu > 10). Największy ≤ 10 to 10 (np. $1 + 4 + 5$).

11 Approx-Subset-Sum (FPTAS)

Strategia: aproksymacyjna (FPTAS). **Współczynnik:** $1 + \varepsilon$, czas wielomianowy w n i $1/\varepsilon$.

Idea: przycinanie list

Dla $0 < \delta < 1$ z listy L tworzymy krótszą L' tak, by dla każdego usuniętego y istniał w L' element z z $\frac{y}{1+\delta} \leq z \leq y$.

Algorytm 8 Trim(L, δ) ($L = (y_1, \dots, y_m)$ rosnąca)

```

1:  $L' \leftarrow (y_1)$ ;  $last \leftarrow y_1$ 
2: dla  $i = 2$  to  $m$  wykonuj
3:   jeżeli  $y_i > last \cdot (1 + \delta)$  to dołącz  $y_i$  do  $L'$ ;  $last \leftarrow y_i$ 
4:   koniec jeżeli
5: koniec dla
6: zwróć  $L'$ 
  
```

Algorytm 9 Approx-Subset-Sum(S, t, ε)

```

1:  $L_0 \leftarrow (0)$ 
2: dla  $i = 1$  to  $n$  wykonuj
3:    $L_i \leftarrow \text{MERGE-LISTS}(L_{i-1}, L_{i-1} + x_i)$ 
4:    $L_i \leftarrow \text{TRIM}(L_i, \varepsilon/(2n))$ 
5:   usuń z  $L_i$  elementy  $> t$ 
6: koniec dla
7: zwróć największy element  $L_n$  (oznaczymy  $z^*$ )
  
```

Współczynnik jakości i złożoność

Twierdzenie 11.1 (FPTAS dla Subset-Sum). *Niech y^* będzie sumą optymalną ($\leq t$), a z^* wynikiem algorytmu. Wtedy $\frac{y^*}{z^*} \leq 1 + \varepsilon$, a czas działania jest wielomianowy w n i $1/\varepsilon$. Algorytm jest więc w pełni wielomianowym schematem aproksymacyjnym (FPTAS).*

Szkic dowodu. Jakość. Indukcyjnie pokazuje się, że dla każdej sumy $y \in P_i$ (pełna lista bez przycinania) istnieje element $z \in L_i$ z $\frac{y}{(1+\delta)^i} \leq z \leq y$, gdzie $\delta = \varepsilon/(2n)$. Dla $i = n$ i sumy optymalnej y^* dostajemy $z^* \geq \frac{y^*}{(1+\varepsilon/(2n))^n}$. Ponieważ $(1 + \frac{\varepsilon}{2n})^n \leq e^{\varepsilon/2} \leq 1 + \varepsilon$ (dla $0 < \varepsilon \leq 1$), mamy $\frac{y^*}{z^*} \leq 1 + \varepsilon$.

Złożoność. Po przycięciu kolejne elementy L_i różnią się czynnikiem $> 1 + \delta$, a należą do $[1, t]$, więc $|L_i| \leq \log_{1+\delta} t + 2 = O(\frac{n \ln t}{\varepsilon})$. Stąd łączny czas $O(\frac{n^2 \ln t}{\varepsilon})$ — wielomianowy w rozmiarze wejścia i $1/\varepsilon$. $\square$

Przykład

$L = (15, 16, 17, 25, 27, 28, 35, 36, 37)$, $\delta = 0,1$. Przycinanie: 15 zostaje; $16 \leq 15 \cdot 1,1 = 16,5$ — usuwamy; $17 > 16,5$ — zostaje; 25; 27, 28 usunięte przez $25 \cdot 1,1 = 27,5$? ($27 \leq 27,5$ usuwamy, $28 > 27,5$ zostaje); itd. Wynik $L' = (15, 17, 25, 28, 35)$ — krótsza lista, każdy usunięty element ma w L' bliskiego reprezentanta od dołu.

12 Maksymalne skojarzenie w grafie dwudzielnym (MM)

Strategia: ścieżki powiększające. **Złożoność:** $O(|V| \cdot |E|)$.

Definicje

$G = (V_1 \cup V_2, E)$ dwudzielnym. *Skojarzenie* $M \subseteq E$: żadne dwie krawędzie nie mają wspólnego końca. Wierzchołek *wolny*, gdy nie jest końcem żadnej krawędzi M . *Ścieżka powiększająca* względem M : ścieżka prosta zaczynająca i kończąca w wierzchołkach wolnych, o krawędziach naprzemiennie spoza M i z M . Wtedy $M' = M \triangle E(P)$ jest skojarzeniem z $|M'| = |M| + 1$.

Algorytm

Graf pomocniczy skierowany G_M : krawędzie spoza M skierowane $V_1 \rightarrow V_2$, krawędzie z M skierowane $V_2 \rightarrow V_1$. Ścieżka powiększająca odpowiada ścieżce skierowanej z wolnego wierzchołka V_1 do wolnego wierzchołka V_2 .

Algorytm 10 APS(G, M) — szukanie ścieżki powiększającej

- 1: $V'_1 \leftarrow$ wolne w V_1 ; $V'_2 \leftarrow$ wolne w V_2 ; zbuduj G_M
 - 2: znajdź (BFS) ścieżkę skierowaną $\vec{P}$ z V'_1 do V'_2 w G_M
 - 3: **jeżeli** istnieje **to zwróć** P w przeciwnym razie zwróć NULL
-

Algorytm 11 MM(G) — maksymalne skojarzenie

- 1: $M \leftarrow \emptyset$
 - 2: **repeat**
 - 3: $P \leftarrow$ APS(G, M)
 - 4: **jeżeli** $P \neq \text{NULL}$ **to** $M \leftarrow M \triangle E(P)$
 - 5: **koniec jeżeli**
 - 6: **until** $P = \text{NULL}$
 - 7: **zwróć** M
-

APS działa w $O(|E|)$; pętla wykonuje się $\leq |V|/2$ razy (każde powiększenie $+1$), więc całość $O(|V| \cdot |E|)$.

Poprawność (twierdzenie Berge’a)

Twierdzenie 12.1 (Berge). *Skojarzenie M jest najliczniejsze w G wtedy i tylko wtedy, gdy w G nie istnieje ścieżka powiększająca względem M . (Twierdzenie zachodzi dla dowolnych grafów, nie tylko dwudzielnych.)*

Dowód. ($\Rightarrow$, w postaci kontrapozycji) Jeśli istnieje ścieżka powiększająca P , to $M' = M \triangle E(P)$ jest skojarzeniem o $|M'| = |M| + 1 > |M|$, więc M nie jest najliczniejsze.

($\Leftarrow$, kontrapozycja) Załóżmy, że M nie jest najliczniejsze; niech M^* będzie skojarzeniem z $|M^*| > |M|$. Rozważmy graf $H = (V, M \triangle M^*)$. Każdy wierzchołek ma w H stopień ≤ 2 (co najwyżej jedna krawędź z M i jedna z M^*), więc spójne składowe H to *ścieżki* i *cykle* o krawędziach naprzemiennie z M i M^* . W każdym cyklu i każdej ścieżce parzystej długości liczby krawędzi z M i M^* są równe. Ponieważ $|M^*| > |M|$, krawędzi z M^* jest w $M \triangle M^*$ więcej niż z M , więc istnieje składowa będąca *ścieżką* z przewagą krawędzi M^* — ścieżka o nieparzystej liczbie krawędzi, zaczynająca i kończąca się krawędzią z M^* . Jej oba końce są wolne względem M (gdyby koniec był pokryty krawędzią M , należałaby ona do tej składowej i przedłużała ścieżkę), a jej krawędzie są naprzemiennie spoza M i z M . Jest to ścieżka powiększająca względem M . $\square$

Twierdzenie 12.2. *Algorytm MM zwraca najliczniejsze skojarzenie.*

Dowód. Każde powiększenie zwiększa $|M|$ o 1 i utrzymuje niezmiennik „ M jest skojarzeniem”. Algorytm kończy się, gdy APS nie znajduje ścieżki powiększającej; z Lematu o poprawności APS oznacza to brak ścieżki powiększającej w G , a z Tw. 12.1 — że M jest najliczniejsze. $\square$

Przykład

$V_1 = \{u_1, u_2, u_3\}$, $V_2 = \{v_1, v_2, v_3\}$, krawędzie $u_1v_1, u_1v_2, u_2v_1, u_3v_2, u_3v_3$. Start $M = \emptyset$. APS znajduje $u_1 \rightarrow v_1$, $M = \{u_1v_1\}$; potem $u_3 \rightarrow v_3$, $M = \{u_1v_1, u_3v_3\}$; potem ścieżka powiększająca $u_2 \rightarrow v_1 \rightarrow u_1 \rightarrow v_2$ (naprzemiennie spoza/z M), po różnicy symetrycznej $M = \{u_2v_1, u_1v_2, u_3v_3\}$ — skojarzenie doskonałe liczności 3. Brak dalszych ścieżek $\Rightarrow$ wynik najliczniejszy.

Część B. Pięć twierdzeń egzaminacyjnych — skorowidz

13 Zestawienie twierdzeń z dowodami

Egzaminacyjne zadanie typu 1 wymaga sformułowania i udowodnienia jednego z pięciu twierdzeń. Pełne dowody znajdują się w odpowiednich sekcjach; poniżej zwięzłe odsyłacze i sformułowania.

- (1) **Struktura problemu szeregowania zadań** (bez dowodu lematu): rodzina zbiorów wolnych od kar tworzy matroid — Tw. 7.3 (sekcja 7). Korzysta z Lematu 7.2 (charakteryzacja $N_t(A) \leq t$), który na egzaminie przyjmujemy bez dowodu.
- (2) **Poprawność algorytmu pary najbliższych punktów**: wystarczy sprawdzić 7 następników w Y' — Tw. 2.1 (sekcja 2).
- (3) **Współczynnik jakości Approx-Vertex-Cover**: algorytm jest 2-aproksymacyjny — Tw. 9.1 (sekcja 9).
- (4) **Współczynnik jakości Greedy-Set-Cover**: algorytm jest $H(\max |S|) \leq (\ln n + 1)$ -aproksymacyjny — Tw. 8.1 (sekcja 8).

- (5) **Twierdzenie Berge’a** o skojarzeniu maksymalnym i ścieżce powiększającej — Tw. 12.1 (sekcja 12).
-

Część C. Projektowanie algorytmów (zadania 3–4)

14 Schemat odpowiedzi i przykłady projektowe

14.1 Schemat poprawnej odpowiedzi

Przy zadaniu „zaprojektuj algorytm” należy podać:

1. **Strategię** (dziel i zwyciężaj / DP / zachłanny / aproksymacyjny) i uzasadnienie wyboru (np. obecność optymalnej podstruktury, NP-trudność $\Rightarrow$ aproksymacja).
2. **Konstrukcję** (podproblem + rekurencja dla DP; reguła wyboru dla zachłannego; podział/scal dla D&Z) i **pseudokod**.
3. **Złożoność** (liczba podproblemów $\times$ koszt; albo rekurencja + Master Theorem).
4. **Poprawność** (optymalna podstruktura + indukcja; lub argument wymiany; lub niezmiennik pętli).
5. **Współczynnik jakości** dla algorytmów przybliżonych (Def. 1.2).

14.2 Przykład A (DP): ważony wybór zajęć

Problem: jak wybór zajęć, lecz zajęcie z_i ma wagę v_i ; maksymalizujemy sumę wag zbioru zgodnego. Zachłanny „najwcześniejszy koniec” tu zawodzi (kontrprzykład: jedno długie zajęcie o dużej wadze). *Rozwiązanie (DP):* sortuj wg t_i ; niech $p(i)$ = największe $j < i$ z $t_j \leq s_i$. Wtedy

$$W[i] = \max(W[i-1], v_i + W[p(i)]), \quad W[0] = 0.$$

Poprawność: optymalne rozwiązanie albo nie zawiera z_i (wtedy $W[i-1]$), albo zawiera z_i i jego reszta jest optymalna na zajęciach kończących się $\leq s_i$ (wtedy $v_i + W[p(i)]$) — optymalna podstruktura. *Złożoność:* $O(n \log n)$ (sortowanie + wyszukiwanie binarne $p(i)$).

14.3 Przykład B (zachłanny + argument wymiany): minimalizacja maksymalnego opóźnienia

Problem: jeden procesor, zadania o czasach t_i i terminach d_i ; minimalizuj $L_{\max} = \max_i (C_i - d_i)$. *Algorytm:* EDF — wykonuj zadania w kolejności niemalejących terminów d_i . *Poprawność (wymiana):* w optymalnym harmonogramie bez „luk” każda inwersja (zadanie o większym terminie przed zadaniem o mniejszym) sąsiednich zadań może być odwrócona bez wzrostu $L_{\max}$; po usunięciu wszystkich inwersji otrzymujemy porządek EDF. *Złożoność:* $O(n \log n)$.

14.4 Przykład C (D&Z): liczba inwersji

Problem: policzyć pary $i < j$ z $a_i > a_j$. *Algorytm:* zmodyfikowany merge sort — podczas scalania, gdy bierzemy element z prawej połowy przed k pozostałymi z lewej, dodajemy k do licznika. *Rekurencja:* $T(n) = 2T(n/2) + O(n) = \Theta(n \log n)$ (Master Theorem). *Poprawność:* inwersje są albo wewnątrz połów (liczone rekurencyjnie), albo „między” połowami (liczone przy scalaniu) — rozłączny podział.

14.5 Przykład D (aproxymacja / FPTAS): problem plecakowy 0/1

Problem: przedmioty o wagach w_i i wartościach v_i , pojemność W ; maksymalizuj wartość przy $\sum w_i \leq W$ (NP-trudny). *FPTAS* analogiczny do Approx-Subset-Sum: skaluj wartości $v'_i = \lfloor v_i/K \rfloor$ z $K = \varepsilon v_{\max}/n$, rozwiąż dokładnie DP po wartościach w czasie $O(n^2 v'_{\max}/1) = O(n^3/\varepsilon)$; otrzymane rozwiązanie ma wartość $\geq (1 - \varepsilon) \text{OPT}$. *Idea dowodu:* zaokrąglenie traci na każdym $z \leq n$ wybranych przedmiotów $< K$, łącznie $< nK = \varepsilon v_{\max} \leq \varepsilon \text{OPT}$.

14.6 Modyfikacje dla szczególnych danych (zadanie 3)

- **Exact-Subset-Sum**, gdy $t = O(\text{poly}(n))$: listy L_i mają $\leq t + 1$ elementów — algorytm *dokładny* i wielomianowy (pseudowielomianowy), bez potrzeby aproksymacji.
- **Para najbliższych punktów na prostej (1D)**: sortuj i porównuj sąsiadów — $O(n \log n)$, faza Połącz sprawdza tylko 1 następnika.
- **Vertex Cover w drzewie/grafie dwudzielnym**: istnieje algorytm *dokładny* wielomianowy (w dwudzielnym: König — min. pokrycie = maks. skojarzenie, sekcja 12); aproksymacja 2 nie jest wtedy potrzebna.
- **Huffman przy częstościach już posortowanych**: zamiast kopca używamy dwóch kolejek FIFO — czas $O(n)$ zamiast $O(n \log n)$.
- **Matrix-Chain dla danych o strukturze (np. wszystkie wymiary równe)**: optimum bywa dane wprost, bez pełnego $\Theta(n^3)$ DP.

Podsumowanie złożoności

Algorytm	Strategia	Złożoność / współczynnik
Para najbliższych punktów	dziel i zwyciężaj	$\Theta(n \log n)$
Matrix-Chain-Order	programowanie dynamiczne	$\Theta(n^3)$ czas, $\Theta(n^2)$ pamięć
Strassen	dziel i zwyciężaj	$\Theta(n^{2.81})$
LCS	programowanie dynamiczne	$\Theta(mn)$
Wybór zajęć	zachłanny	$O(n \log n)$
Huffman	zachłanny	$O(n \log n)$
Szeregowanie (SPT)	zachłanny (wymiana)	$O(n \log n)$
Szeregowanie z karami	zachłanny (matroid)	$O(n^2)$
Greedy-Set-Cover	aproksymacyjny	$H(\max S) \leq \ln n + 1$
Approx-Vertex-Cover	aproksymacyjny	współczynnik 2, $O(V + E)$
Exact-Subset-Sum	DP (listy)	wykl.; wielom. gdy t wielom.
Approx-Subset-Sum	FPTAS	$1 + \varepsilon$, $O(n^2 \ln t/\varepsilon)$
MM (skojarzenie dwudzielne)	ścieżki powiększające	$O(V \cdot E)$